

LGD File Documentation

Overview

The LGD compiler is an offline static compiler + static linker.

The relationship between LGC and LGD can be analogized to that of a Java source file and a JVM-compiled `.class` file.

It was originally designed for early mobile game platforms. Its purpose is to pre-compile readable C-like scripts (`.lgc`) into machine-executable binary bytecode (`.lgd`) in exchange for faster loading speeds and lower CPU overhead. Encryption is merely an incidental byproduct.

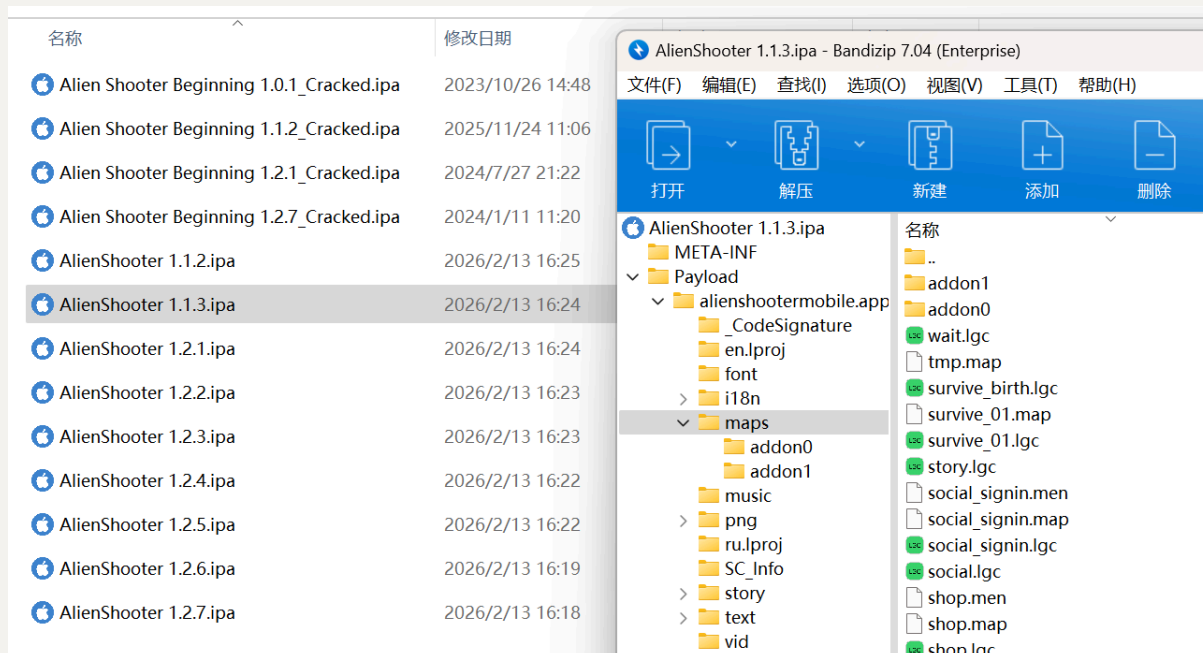
Suppose a level script `Level01.lgc` references 10 external scripts, such as `common.lgc`. Without compilation, the engine would need to simultaneously initiate 10 file I/O operations for reading and parsing when loading the level, which is highly inefficient. However, after using the LGD compiler, all dependencies are packaged and linked into a single independent `.lgd` file. The engine only needs to handle a single I/O operation to complete the loading process, significantly improving execution efficiency.

Its compilation method mainly consists of three parts:

- **Preprocessing:** Recursively processes `#include` directives in the source code. It "kneads" all referenced common scripts (such as `items.lgc`, `common.lgc`) into a final binary stream, much like linking C code.
- **Syntax Validation:** Features complete syntax validation capabilities; if there is an error, it will directly report it.
- **Map Binding Mechanism:** The compilation process uses a `.map` file as an index. It adopts a "one map, one script package" strategy.

Note

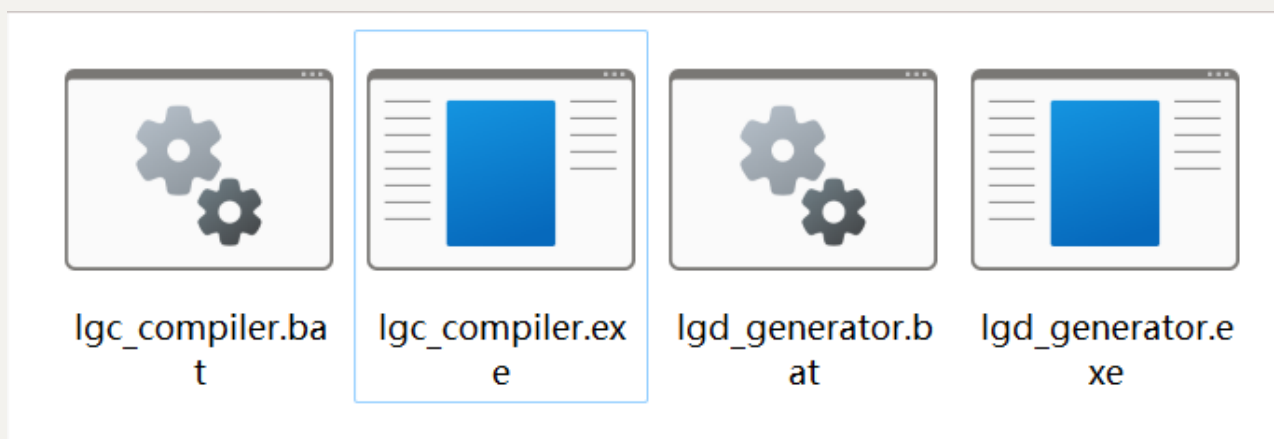
The last version to use lgc was Alien Shooter The Beginning 1.1.3 IOS VER, released on November 22, 2013.



Version and Usage Instructions

Currently, we have two versions of the compiler on hand (Sigma's confidentiality is extremely strict, but they occasionally gift us some developer tools. I suspect this might also be related to the publisher 8-floor-games ; quite a few good things were leaked from versions collaborated with them, so Sigma terminated their partnership around 2022-2023). For convenience, let's call them the New Version and the Old Version. The main analysis in this article is based on the New Version.

- Old version `lgd_generator.exe` compilation time: 2013-12-14 14:39:05
- New version `lgc_compiler.exe` compilation time: 2017-10-31 08:36:40



There is no fundamental difference in how they run; the new version just adds a lot of weird functions, though there is a high probability you will never need them.

The new engine version is backward-compatible with LGDs generated by the old version.

Old Version Parameters

PARAMETER	DESCRIPTION
<code>all</code>	Compiles all lgc files in the current directory and all subdirectories, generating lgd files.
<code>[filename]</code>	Generates an lgd only for the specified single lgc file.

New Version Parameters

PARAMETER	FORMAT	LOGICAL DESCRIPTION
1. Source Directory	<code>--source-dir=DIR</code>	Specifies the root path to search for <code>.lgc</code> files. If not set, defaults to the current directory.
2. Target Directory	<code>--target-dir=DIR</code>	Specifies the storage location for the generated <code>.lgd</code> files, retaining the original directory structure.
3. Macro Definition	<code>-D<DEFINE></code>	Sets preprocessor macros (e.g., <code>-DPLATFORM_WIN=1</code>), used for conditional compilation inside the script.
4. Include Path	<code>-I<PATH></code>	Adds an <code>#include</code> search path. Supports overlapping multiple paths.
5. Verbose Output	<code>-v</code> or <code>--verbose</code>	Enables detailed log mode, displaying symbol processing details during compilation.
6. Base Expansion Info	<code>--base-expansion-info PATH</code>	Patch? Points to an existing expansion info file; if it does not exist, it will attempt to create one.
7. Expansion File	<code>--expansion-file=PATH</code>	Patch? Specifies the path to an expansion/patch file, triggering Checksum calculation logic.
8. Version Number	<code>--version=VERSION</code>	Specifies a version string for the patch, stored in <code>EXPANSION_VERSION</code> .
9. Error Handling	<code>--stop-on-error</code>	Stops immediately upon encountering the first compilation error, skipping subsequent files.

PARAMETER	FORMAT	LOGICAL DESCRIPTION
10. Target Files	<code>FILES</code>	(Positional argument) Specifies one or more specific <code>.lgc</code> files to compile.

One more thing: when using `include` in the old version, you must add the `maps/` path.

Warning

Regarding `#include` path differences between versions:

- **Old version compiler must** explicitly include the `maps/` relative path prefix when referencing dependencies:

```
1 #include "maps\core\export.lgc"
2 #include "maps\common.lgc"
3 #include "maps\common_campaign.lgc"
```

- **New version compiler strictly forbids** carrying the `maps/` path prefix and must point directly to the subdirectory or relative path:

```
1 #include "core\export.lgc"
2 #include "common.lgc"
3 #include "common_campaign.lgc"
```

File Format

Before analyzing, let's open a random file to look at its format, making subsequent analysis easier.

You can see:

Part 1: Very sparse, basically consisting of null bytes (empty text).

tutorial_00.lgd x																	0123456789ABCDEF
	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F	
0:0000	57	07	00	00	82	00	01	00	00	00	00	00	00	00	82	00	w...
0:0010	01	00	00	00	00	00	00	00	82	00	01	00	00	00	00	00	
0:0020	00	00	82	00	01	00	00	00	00	00	00	00	82	00	01	00	
0:0030	00	00	00	00	00	00	8A	00	01	00	00	00	00	00	00	00	
0:0040	8A	00	01	00	00	00	00	00	00	00	82	00	01	00	00	00	
0:0050	00	00	00	00	82	00	01	00	00	00	00	00	00	00	82	00	
0:0060	01	00	00	00	00	00	00	00	82	00	01	00	00	00	00	00	
0:0070	00	00	8A	00	01	00	00	00	FF	FF	FF	FF	82	00	01	00	
0:0080	00	00	00	00	00	00	82	00	01	00	00	00	00	00	00	00	
0:0090	82	00	01	00	00	00	00	00	00	00	8A	00	01	00	00	00	
0:00A0	00	00	00	00	82	00	01	00	00	00	00	00	00	00	82	00	
0:00B0	01	00	00	00	00	00	00	00	82	00	01	00	00	00	00	00	
0:00C0	00	00	82	00	01	00	00	00	00	00	00	00	82	00	01	00	
0:00D0	00	00	00	00	00	00	82	00	01	00	00	00	00	00	00	00	
0:00E0	00	00	01	00	00	00	00	00	00	00	01	00	01	00	00	00	

Part 2: Relatively sparse, with a few garbled characters visible.

tutorial_00.lgd x																	0123456789ABCDEF
	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F	
0:4970	00	01	00	00	00	00	00	00	00	36	03	00	00	64	63	76	6...dcv
0:4980	74	7C	58	75	7D	72	74	63	00	02	00	00	00	64	00	00	t Xu}rtc.....d..
0:4990	00	00	00	00	00	01	00	00	00	00	00	00	00	00	00	00	
0:49A0	00	54	65	72	76	63	72	44	67	65	7E	63	72	00	02	01	.TervcrDge~cr...
0:49B0	00	00	41	00	00	00	01	00	00	00	06	00	00	00	00	00	..A.....
0:49C0	00	00	00	00	00	00	51	7B	76	70	7A	76	79	00	02	01	Q{vpzvy...
0:49D0	00	00	42	00	00	00	07	00	00	00	01	00	00	00	00	00	..B.....
0:49E0	00	00	00	00	00	00	44	67	72	74	7E	76	7B	53	76	7A	Dgrt~v{Svz
0:49F0	76	70	72	54	78	72	71	71	00	02	00	00	00	43	00	00	vprTxrqq.....C..
0:4A00	00	08	00	00	00	02	00	00	00	00	00	00	00	00	00	00	
0:4A10	00	51	7E	65	64	63	42	79	7E	63	00	02	01	00	00	44	.Q~edcBy~c.....D
0:4A20	00	00	00	0A	00	00	00	02	00	00	00	00	00	00	00	00	
0:4A30	00	00	00	59	72	6F	63	42	79	7E	63	00	02	01	00	00	...YrocBy~c.....
0:4A40	45	00	00	00	0C	00	00	00	00	00	00	00	00	00	00	00	E.....
0:4A50	00	00	00	00	50	72	63	44	67	65	7E	63	72	00	02	00	PrcDge~cr...
0:4A60	00	00	46	00	00	00	0C	00	00	00	04	00	00	00	00	00	..F.....
0:4A70	00	00	00	00	00	00	50	72	63	44	67	65	7E	63	72	44	PrcDge~crD
0:4A80	74	65	00	02	00	00	00	47	00	00	00	10	00	00	00	03	te.....G.....
0:4A90	00	00	00	00	00	00	00	00	00	00	00	51	7E	79	73	59	Q~ysY
0:4AA0	72	76	65	72	64	63	44	67	65	7E	63	72	00	02	01	00	rverdcDge~cr....
0:4AB0	00	48	00	00	00	13	00	00	00	05	00	00	00	00	00	00	.H.....
0:4AC0	00	00	00	00	00	51	7E	79	73	59	72	76	65	72	64	63	Q~ysYrverdc
0:4AD0	44	67	65	7E	63	72	24	53	00	02	00	00	00	E5	00	00	Dge~cr\$S.....
0:4AE0	00	10	00	00	00	06	00	00	00	00	00	00	00	00	00	00	

Part 3: FF padding (strictly speaking, there are two sub-parts here).

tutorial_00.lgd x																																
	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0:C100	00	03	02	00	00	CC	20	01	00	53	07	00	00	00	00	00																
0:C110	00	00	00	00	00	00	00	00	00	FF	FF	FF	FF	FF	FF	FF																
0:C120	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF																
0:C130	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF																
0:C140	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF																
0:C150	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF																
0:C160	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF																
0:C170	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF																
0:C180	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF																
0:C190	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF																
0:C1A0	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF																
0:C1B0	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF																
0:C1C0	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF																
0:C1D0	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF																
0:C1E0	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF																
0:C1F0	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF																
0:C200	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF																
0:C210	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF																
0:C220	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF																

Part 4: Dense, where you can see a lot of % and other messy bytes.

tutorial_00.lgd x																																
	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0:C900	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF																
0:C910	FF	FF	FF	FF	FF	FF	FF	FF	FF	D9	24	01	00	01	6F	24																
0:C920	25	25	24	44	25	25	25	34	09	35	25	25	25	01	6F	24	%%\$D%%4.5%%.o\$															
0:C930	25	25	24	5F	25	25	25	37	30	3D	3F	25	25	25	01	6F	%%\$_%%70=?%%.o															
0:C940	24	25	25	24	05	25	25	25	2C	3A	3C	E0	26	25	25	39	%%\$\$.%%%,< .%%9															
0:C950	2A	25	25	25	01	6F	24	25	25	3A	3C	E2	26	25	25	3A	*%%.o\$%%:< .%%:															
0:C960	3C	E8	26	25	25	01	6E	24	25	25	01	69	24	25	25	01	< .%%.n\$%%.i\$%%.															
0:C970	68	24	25	25	01	6B	24	25	25	01	20	25	25	25	01	23	h\$%%.k\$%%. %%%.#															
0:C980	25	25	25	64	03	74	24	25	25	3C	EB	26	25	25	01	74	%%d.t\$%%< .%%.t															
0:C990	24	25	25	24	5D	25	25	25	00	6A	24	25	25	01	09	25	%%\$]\$%%.j\$%%..%															
0:C9A0	25	25	01	08	25	25	25	6A	3C	EA	26	25	25	01	74	24	%%..%%j< .%%.t\$															
0:C9B0	25	25	24	7A	25	25	25	01	75	24	25	25	01	09	25	25	%%\$z%%.u\$%%..%															
0:C9C0	25	01	08	25	25	25	6A	3C	F5	26	25	25	01	74	24	25	%%..%%j<.&\$%%.t\$%															
0:C9D0	25	3A	3C	F4	26	25	25	3A	3C	F2	26	25	25	01	77	24	%:< .%%:< .%%.w\$															
0:C9E0	25	25	01	76	24	25	25	01	71	24	25	25	01	70	24	25	%%.v\$%%.q\$%%.p\$%															
0:C9F0	25	01	20	25	25	25	01	23	25	25	25	64	03	7D	24	25	% .%%. #%%d.}%\$%															
0:CA00	25	3C	FD	26	25	25	01	7D	24	25	25	24	5E	25	25	25	%<.&\$%%.}%\$%^%%%															
0:CA10	00	73	24	25	25	01	09	25	25	25	01	08	25	25	25	6A	.s\$%%..%%..%%j															

Part 1: Literal Table

It is primarily responsible for recording the status and initial values of all variables -- (global variables, function parameters, local variables).

The fixed first 4 bytes represent the index count.

Important

1. The engine's lowest level actually only supports `int`, `string`, and their corresponding arrays.
2. Although you can see the `boolean` type in the scripts, they are essentially `int` at their core. The author used macro definitions to make the scripts look closer to C code.
3. Although the engine reserves support for "pointer" operations in the underlying flags, they have never been actually used in official scripts.
4. There is no difference in the compiled file between `static` and non-`static` **global variables**; the difference between these two keywords only manifests when they are used as **local variables**.

1. Core Concept: The Difference Between Index and Global ID

To understand the Literal Table, you must first clarify the difference between `Index` (Entry Index) and `Global ID` (Global/Logical ID). This is also the most confusing part of the entire table:

- **Index (Entry Index):** Represents a physical storage "slot". A basic variable takes up 1 Index; for an array of length N , each of its elements will occupy 1 independent Index. The `Entry Count` at the beginning of the file refers to the total number of Indexes.
- **Global ID (Logical Group ID):** Represents logical "ownership/affiliation". It is assigned in groups:
 - A standard variable exclusively occupies 1 ID.
 - **An entire array, regardless of how many elements it has, shares the same ID.**
 - **A function, along with all its internal parameters and local variables, also entirely shares the same ID.**

👉 Experimental Comparison: Array Index vs. ID Allocation

Let's take an `int` array with 3 elements as an example:

```
1 int intArray[3] = { 114, 514, 19198 };
```

In the Literal Table, it is split into 3 Index entries, but they share the same ID (for example, ID is 9). Please observe the difference between the first array item and the subsequent items:

```
1 // [GlobalID: 9] Occupies 3 Index entries
2 // Flag | String (Empty) | Size or ID | Value
3 8E 00 03 00 00 00 /* Size: 3 */ 72 00 00 00 /*
   val: 114 */
4 8E 00 09 00 00 00 /* ID: 9 */ 02 02 00 00 /*
   val: 514 */
5 8E 00 09 00 00 00 /* ID: 9 */ FE 4A 00 00 /*
   val: 19198 */
```

💡 Tip

The first Index entry of the array records the **total length of the array (Size)**; while all subsequent Index entries of the array record the **Global ID** they belong to in the same position. Amazing, isn't it? Haha 😊.

2. Data Entry Structure and Flag Bits

Through the above analysis, we can summarize the standard memory structure for each Index entry as follows:

OFFSET (BYTE)	FIELD NAME	LENGTH	PARSING DESCRIPTION
0	Flag	1 byte	Variable status/type flag (see table below for details).
1 ~ n+1	String Content	n + 1 bytes	Variable name or string content, ending with 00. If there is no string, it is just a single 00. (Note: Content is encrypted via XOR 17)

OFFSET (BYTE)	FIELD NAME	LENGTH	PARSING DESCRIPTION
n+2 ~ n+5	ID / Size	4 bytes	Core Distinguishing Field: If it is an independent variable or the first item of an array, this records the Size ; if it is a subsequent element of an array, this records the ID . If it is a method/EXTERN, it records each parameter and local variable in the form of variables.
n+6 ~ n+9	Value	4 bytes	The initial value for a signed integer. If it is a string type, this is usually padded with zeros as 00 00 00 00.

Flag Bit (Byte 0) Composition Logic:

The Flag value is actually formed by a bitwise OR combination of attributes with different weights:

WEIGHT (HEX)	FUNCTION CATEGORY	MEANING DESCRIPTION
0x80	Base Bit	Literal Entry (Mandatory identifier for Part 1 literal values)
0x01	Type Bit	Belongs to String (string)
0x02	Type Bit	Belongs to Int (integer)
0x04	Structure Bit	Belongs to Array (array [])
0x08	Status Bit	Initialized (assigned and initialized)
0x20	Pointer Bit	Pointer (*, unused in official scripts)
0x40	(Old Ver) Status	Uninitialized (unassigned/uninitialized)

Based on the weight combinations above, common Flag constants in decompilation are as follows:

- **Basic Unassigned:** 81 (unassigned string), 82 (unassigned int), 85 (unassigned string[]), 86 (unassigned int[])
- **Basic Assigned:** 89 (assigned string), 8A (assigned int), 8D (assigned string[]), 8E (assigned int[])
- **Rare Pointer Types:** A1 (string), A2 (int), A5 (string[]), A6 (int[])

Old Version Flags:

- **Basic Unassigned:** C1 (unassigned string), C2 (unassigned int), C5 (unassigned string[]), C6 (unassigned int[])
 - **Basic Assigned:** 89 (assigned string), 8A (assigned int), 8D (assigned string[]), 8E (assigned int[])
 - **Rare Pointer Types:** E1 (unassigned string), E2 (*unassigned int*), E5 (unassigned string[]), E6 (*unassigned int[]*)
-

3. Basic Variable Bytecode Parsing

To intuitively understand the structure of the Literal Table, this section will analyze the hexadecimal data of various types. Arrays have already been discussed earlier, so they are omitted here.

Case 1: Basic Int Variable

We define an unassigned `unusedValue` and a `gval` assigned the value of 166:

```
1 int unusedValue;  
2 int gval = 166;
```

The corresponding hexadecimal data in the Literal Table is as follows:

```
1 // [GlobalID: 0] variable: int unusedValue;  
2 // Flag | Str | Size(4 bytes)      | value(4 bytes)  
3 82      00      01 00 00 00 /* Size: 1 */    00 00 00 00 /* val:  
  0 */  
4  
5 // [GlobalID: 1] variable: int gval = 166;  
6 // Flag | Str | Size(4 bytes)      | value(4 bytes)  
7 8A      00      01 00 00 00 /* Size: 1 */    A6 00 00 00 /* val:  
  166 (0xA6) */
```

Analysis:

- **Flag:** 82 represents an unassigned int; 8A represents an assigned int.
- **Str:** Because it's an integer variable, there is no string content; it just uses 00 as a placeholder and ends.
- **Size:** This is a single variable, so the Size is fixed at 1 (01 00 00 00).
- **Value:** unusedValue is filled with 0 by default; gval's 166 converted to hexadecimal is A6 00 00 00.

Case 2: Basic String Variable

String handling is similar to integers, but its "value" is written directly into the **Str** field (and has undergone simple XOR 17 encryption), while the trailing **value** field is forced to zero.

```
1 string unusedTxt;
2 string txt = "sample text";
```

The corresponding hexadecimal data in the Literal Table is as follows:

```
1 // [GlobalID: 4] variable: string unusedTxt;
2 // Flag | Str | Size(4 bytes) | value(4 bytes)
3 81 00 01 00 00 00 /* Size: 1 */ 00 00 00 00 /* val:
  0 */
4
5 // [GlobalID: 5] variable: string txt = "sample text";
6 // Flag | Str (Encrypted String) |
  Size(4 bytes) | value(4 bytes)
7 89 64 76 7A 67 7B 72 37 63 72 6F 63 00 /* "sample text" */
  01 00 00 00 /* Size: 1 */ 00 00 00 00 /* val: 0 */
```

Analysis:

- **Flag:** 81 represents an unassigned string; 89 represents an assigned string.
- **Str:** When unassigned, it is only 00. When assigned "sample text", the engine encrypts it via XOR 17 and stores it (hexadecimal 64 76...), padding 00 at the end as the string terminator.
- **Value:** Since the data already exists in the preceding **Str** block, the subsequent

Value field is uniformly padded with 00 00 00 00.

Case 3: Regular Method

```
1 ScriptEvent4( int unit, int var1 = 114, string var2 = "text" ) {  
2     int val = 100;  
3 }
```

```
1 // [GlobalID: 0] -----  
2 // Function: ScriptEvent4( int para1, int para2, string para3 )  
3 {  
4     // (Locals inside this block...)  
5     // [Arg 1]  
6     82 00                                01 00 00 00 /* Para 1 */      00  
7     00 00 00 /* val: 0 */  
8     // [Arg 2]  
9     8A 00                                01 00 00 00 /* Para 2 */      72  
10    00 00 00 /* val: 114 */  
11    // [Arg 3]  
12    89 63 72 6F 63 00 /* "text" */      01 00 00 00 /* Para 3  
13    */      00 00 00 00 /* val: 0 */  
14    82 00                                01 00 00 00 /* Para 4 */      00  
15    00 00 00 /* val: 0 */
```

As mentioned earlier, a method shares a single **Global ID**, which records all of its parameters and local variables.

Case 4: EXTERN Method

The `extern` keyword is used to declare built-in functions provided by the engine's underlying layer. In the Literal Table, `extern` functions behave very similarly to normal functions: they are also assigned an independent **Global ID**, and their parameters are recorded.

```
1 extern MessageText(string text, int x, int y, int fontNvid = -1)  
   114;
```

```

1 // Extern: MessageText (Extern ID: 114)
2     81 00                                01 00 00 00 /* Size: 1 */    00
    00 00 00 /* string text */
3     82 00                                01 00 00 00 /* Size: 1 */    00
    00 00 00 /* int x */
4     82 00                                01 00 00 00 /* Size: 1 */    00
    00 00 00 /* int y */
5     8A 00                                01 00 00 00 /* Size: 1 */    FF
    FF FF FF /* int fontNvid = -1 */

```

4. Compiler Features

Feature 1: Complete Erasure of Constants

Can't find macro definitions or constants in the table? That's because the compiler directly "discards" them during the preprocessing stage and replaces them with literal numeric values.

This is quite normal; many compilers have this feature.

```

1 // Source code
2 #define USED_CONST 150
3 int constVal = USED_CONST;

```

```

1 // Compiled Literal Table: Handled directly as int constVal =
  150;
2 // Flag | Str | Size | Value
3     8A    00    01 00 00 00 /* Size: 1 */    96 00 00 00 /* val:
  150 */

```

Feature 2: Differentiated Handling of Local `Static` Variables

The logic for handling local variables inside functions is completely different depending on whether they are `static` or not, especially for arrays, which is extremely bizarre.

If it is a `static` local variable: It will directly retain its initialized value in the Literal Table.

```
1 static int stval = 100;
2 static int starr[] = {0x2222, 0x3333, 0x4444, 0x5555};
```

```
1 // values are normally retained (64 00 00 00 is 100, 22 22 00 00
  is 0x2222...)
2 8A 00 01 00 00 00 /* Para 4 */ 64 00 00 00 /* val: 100 */
3 8E 00 04 00 00 00 /* Para 5 */ 22 22 00 00 /* val: 8738
  (0x2222) */
4 // ...subsequent array elements normally carry their values
```

If it is a `non-static` local variable: The compiler will forcefully downgrade it to an **uninitialized variable**, and its initial values in the Literal Table are all filled with 0. The actual assignment operation is deferred by the compiler to the later Bytecode (bytecode) execution stage!

```
1 int val = 100;
2 int arr[] = {0x2222, 0x3333, 0x4444, 0x5555};
```

```
1 // Flags become 82 (unassigned int) and 86 (unassigned int
  array), values are all 00 00 00 00!
2 82 00 01 00 00 00 /* Para 9 */ 00 00 00 00 /* val: 0 */
3 86 00 04 00 00 00 /* Para 10 */ 00 00 00 00 /* val: 0 */
4 86 00 11 00 00 00 /* Para 11 */ 00 00 00 00 /* val: 0 */
5 // ...all are 0
```

Part 2: Symbol Table Structure Analysis

Immediately following the Literal Table is the **Symbol Table**. Its core purpose is to bind the "variable names" and "function names" defined in our source code to the `Global IDs` allocated in Part 1, and to record their metadata (such as entry offsets, parameter counts, etc.).

💡 *Engine Trivia: Where did the local variables go?*

*In the Symbol Table, you will find that only **global variables** and **function names** exist. All function parameter names and local variable names are entirely discarded by the compiler! From the perspective of the engine's underlying layer, local variables are nothing more than a bunch of unnamed stack offsets and the **Indexes** they occupy in the Literal Table.*

1. Overall Symbol Table Structure

The Symbol Table starts with a 4-byte count (Entry Count), indicating how many Global ID entries there are in total in the table.

Following that is the sequentially arranged entry data. Each entry consists of a **dynamically length string** + a **fixed 24-byte attribute block**:

- **Symbol Name (String):** A string encrypted via XOR 17, ending with **00**.
- **Attribute Block (24 bytes):** A binary structure of fixed length, specifically defined as in the table below.

OFFSET (BYTE)	LENGTH	FIELD NAME	DETAILED MEANING AND PROCESSING LOGIC
+0	2 bytes	Type	Symbol Type and Status Flags. Distinguishes whether it is a variable, a regular function, or an Extern function (see the status table below for details).
+2	2 bytes	Meta / Garbage	Uninitialized stack residual garbage data

OFFSET (BYTE)	LENGTH	FIELD NAME	DETAILED MEANING AND PROCESSING LOGIC
+4	4 bytes	Extern ID / Offset / Padding	Core Multiplexed Field (Built-in system ID / Relative bytecode segment offset). • Extern Function (Type=02): Records its corresponding engine underlying system function ID. • Regular Function (Type=03): Records this function's Relative Bytecode Offset (Relative Offset) within Part 4 (Bytecode segment). (Formula: <i>Physical file offset = Bytecode segment starting offset + Relative bytecode offset = actual entry address in decompiled code</i>). • Global Variable (Type=01): No substantive effect, fixed padding of 00 00 00 00.
+8	4 bytes	P1_Idx	Literal Table Index (Part 1 Index). Points to the starting position of this symbol's corresponding initial value in Part 1 (the Literal Table).
+12	4 bytes	Size	Size / Element Count. • Variable: The length of the array (e.g., 1 for <code>int a</code>, 3 for <code>int a[3]</code>). • Function: The total length of the parameter list (Note: Does NOT include local variables).
+16	4 bytes	File_ID	Source File ID. • Allocation Mechanism: Strictly increments from 0. 0 represents the main file, and 1, 2... are assigned to each included file (<code>#include</code>) in the chronological order of their first physical opening during preprocessing expansion. • Gaps/Jumps Explanation: Non-contiguous <code>File_IDs</code> (e.g., 0, 12, 13...) often appear in the symbol table because the skipped files in between (e.g., 1~11) did not define any global variables, producing no symbol table entries and causing apparent gaps. • Note: Only global variables have a non-zero <code>File_ID</code>; other entries are always 0.

OFFSET (BYTE)	LENGTH	FIELD NAME	DETAILED MEANING AND PROCESSING LOGIC
+20	4 bytes	Run_ID	Local Slot Index during Compilation (Run_ID). Has no substantive role during decompilation or runtime. • File Locality: It is the independent local slot index for global variables within their respective source file (<code>File_ID</code>). Since different source files increment this independently, duplicate <code>Run_ID</code>s will occur across different files. • Non-variable Reset: Only global variable symbols have non-zero values here; all regular functions and external functions (Extern) have this value set to 0. • Reverse Engineering Conclusion: The engine addresses memory entirely via the globally unique <code>P1_Idx</code> when executing bytecode. <code>Run_ID</code> is purely a compilation artifact and can be completely ignored during reverse analysis.

2. Type Flag and "Return Value Tracking" Mechanism

The `Type` field at offset +0 of the Symbol Table is not just used to distinguish between variables and functions; it hides a highly ingenious mechanism: **The compiler statically analyzes whether the return value of a function is actually used in the context and records this in the Type.**

Complete List of Type Flag Enumerations:

- `01 00` : Global Variable
- `02 00` : External Function (Extern), and return value is **unused**
- `02 01` : External Function (Extern), and return value is **used**
- `03 00` : Regular Function (Function), no return value (Void)
- `03 01` : Regular Function (Function), has return value, and is used by the `await` keyword
- `03 02` : Regular Function (Function), has return value, but is **unused**
- `03 03` : Regular Function (Function), has return value, and is **used**

👉 Experimental Comparison: Extern Return Value Tracking

Taking the extern function `CreateSprite` as an example, different calling methods will directly change its `Type` signature in the Symbol Table:

```
1 // Scenario A: Just calling it, ignoring the return value
2 CreateSprite(1, 10, 20, 0);
3 // Corresponding Type -> 02 00
4
5 // Scenario B: Assigning the return value to a variable
6 int s = CreateSprite(1, 10, 20, 0);
7 // Corresponding Type -> 02 01
```

3. The True Meaning of Size: The Boundary Between Parameters and Locals

For variables, `Size` is very easy to understand (it's just the array length); but for functions, the `Size` field **only counts the number of function parameters, completely ignoring internal local variables**.

👉 Experimental Comparison: Function Size Counting Logic

We define the following code:

```
1 ScriptEvent4( int unit, int var1 = 114, string var2 = "text" ) {
2     static int stval = 100;
3     static int starr[] = {0x2222, 0x3333, 0x4444, 0x5555};
4     int val = 100;
5     int arr[] = {0x2222, 0x3333, 0x4444, 0x5555};
6 }
7
8 int afterFun[] = {2, 3, 4};
```

Analyzing the compiled Symbol Table entries:

1. Function `ScriptEvent4`:

```

1 // Encrypted Name: ScriptEvent4
2 44 74 65 7E 67 63 52 61 72 79 63 23 00
3 03 00 // Type: Function (Void - no return value)
4 C9 00 // Meta
5 00 00 00 00 // Padding: 0
6 14 00 00 00 // P1_Index: 20
7 03 00 00 00 // Size: 3 (Core verification point: Only counts
the 3 parameters unit, var1, var2)
8 00 00 00 00 // FileID: 0
9 00 00 00 00 // RunID: 0

```

2. Global Variable `afterFun`:

```

1 // Encrypted Name: afterFun
2 76 71 63 72 65 51 62 79 00
3 01 00 // Type: Variable
4 CA 00 // Meta: Residual garbage data
5 00 00 00 00 // Padding: 0
6 21 00 00 00 // P1_Index: 33
7 03 00 00 00 // Size: 3 (Array length is 3)
8 00 00 00 00 // FileID: 0
9 1A 00 00 00 // RunID: 26

```

As you can see, although `ScriptEvent4` is internally stuffed with local variables like `stval`, `starr`, `val`, taking up a large amount of `P1_Index` space, the `Size` in the Symbol Table still displays as `03`. This confirms that the Symbol Table manages functions at the "external calling signature" level and does not care about their internal local states. However, we can still estimate the local variables inside a function by using the index difference between the adjacent upper and lower groups and combining it with `P1_Idex`.

Part 3: Special Value Table (Action & ScriptEvent Table)

An area that defaults to being entirely padded with `FF`, used for `ActionX` and `ScriptEventX`, likely for quick positioning and map interaction.

The size and structure of this Special Value Table are **absolutely fixed**, with a total size of **2048 Bytes**. The calculation formula is: `4 bytes (size of a single slot) × 256 (supported IDs 0~255) × 2 (two groups: variables and functions)`.

If these variables or functions with special numbers are defined in the `.lgc` script, the compiler will "hollow out" the `FF FF FF FF` at the corresponding index in this area during compilation, and fill in their reference addresses.

Sample Code

```
1  int arr[] = {1,2,3}; //id = 0 index = 0-3
2  int a = 114; //id = 1 index = 4
3
4  int Action2 = 100; //id = 2 index = 5
5  int Action5 = 200; //id = 3 index = 6
6
7  int arr2[] = {1,2,3}; //id = 4
8
9  ScriptEvent1( int val1, int val2, int val3 ){
10     //id = 5
11 }
12
13 ScriptEvent3( int val1, int val2, int val3 ){
14     //id = 6
15 }
```

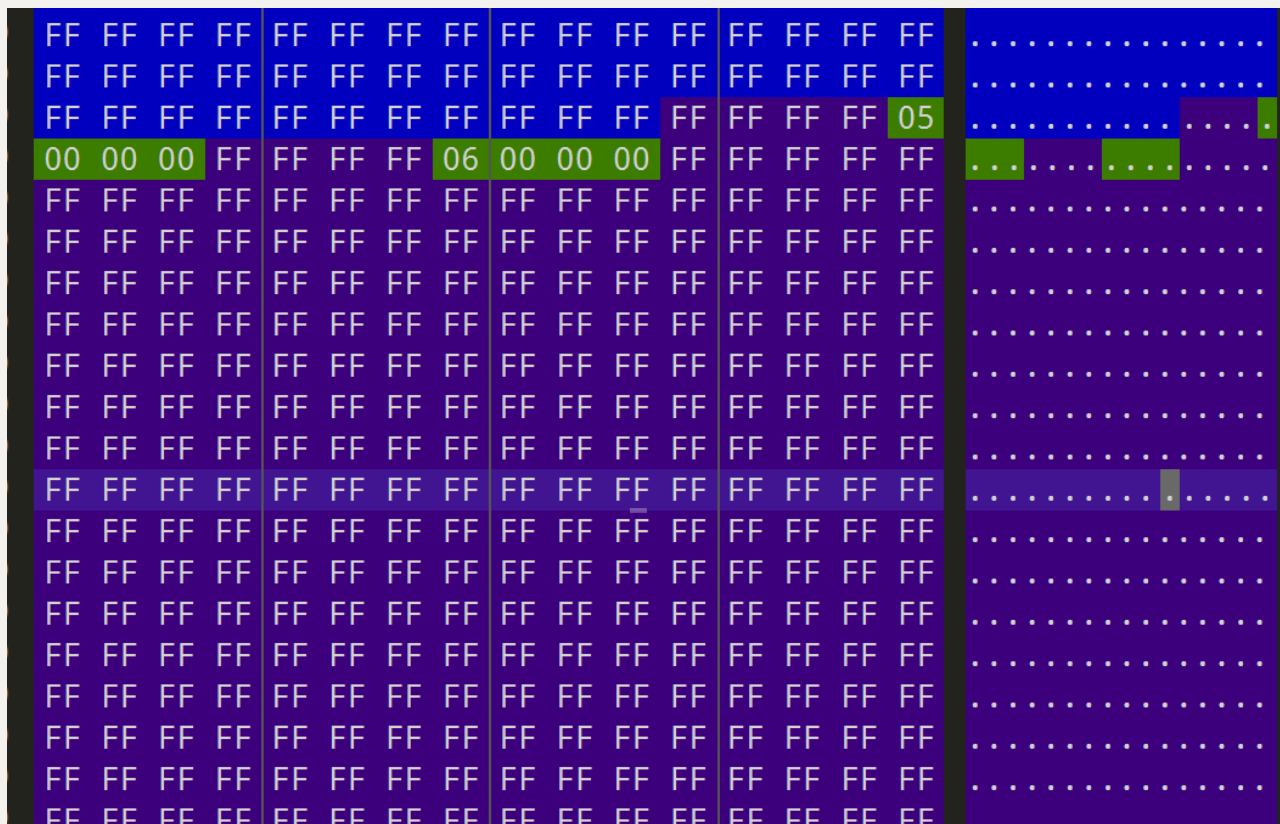
Rule A: Special Variable `int ActionX`

- **Type Restriction:** Must be of `int` type.
- **Filled Content:** Records this variable's **P1_Index (Entry Index)** in the Literal Table (Part 1).
- **Example:** As shown in the code, if Action2/Action5 are defined, then the corresponding `FF` padding is hollowed out and filled with the index value.

0150	00	00	00	00	00	00	44	74	65	7E	67	63	52	61	72	79	Dte~gcRar
0160	63	24	00	03	00	68	00	01	00	00	00	0C	00	00	00	03	c\$...h.....
0170	00	00	00	00	00	00	00	00	00	00	00	FF	FF	FF	FF	FF	
0180	FF	FF	FF	04	00	00	00	FF	FF	FF	FF	FF	FF	FF	FF	05	
0190	00	00	00	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	
01A0	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	
01B0	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	
01C0	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	
01D0	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	
01E0	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	
01F0	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	
0200	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	
0210	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	
0220	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	
0230	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	
0240	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	
0250	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	
0260	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	
0270	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	

Rule B: Special Function `ScriptEventX`

- **Mandatory Parameter Restriction:** Must contain **and only contain 3 parameters**, for example, `ScriptEvent5(int var1, int var2, int var3)`. If the number of parameters is incorrect, compilation will fail.
- **Filled Content:** Records this function's **Global ID** in the Literal Table.
- **Example:** As shown in the code, if `ScriptEvent1/ScriptEvent3` are defined, then the corresponding `FF` padding is hollowed out and filled with the Global ID value.



Part 4: Bytecode Segment and Instruction Set (Opcode Table)

The beginning of the code segment consists of a **4-byte integer**, representing the total length of the subsequent bytecode.

These first 4 bytes are in **plaintext** (unencrypted), but all bytecode data starting from the 5th byte is entirely encrypted via **XOR 0x25**.

Instruction Set

1. Value Types & Constants

OPCODE	MNEMONIC	SYMBOL	DESCRIPTION	OPERANDS (SIZE & TYPE)
0x01	PUSH_INT		Pushes an integer constant value to the stack.	4 Bytes (32-bit integer constant value)
0x02	PUSH_STR		Pushes a string constant value to the stack.	Variable length (String ending in 0x00)

2. Unary Operators

These operators act directly on the current data at the top of the stack, so the instructions themselves carry no operands (0 Bytes).

OPCODE	MNEMONIC	SYMBOL	DESCRIPTION	OPERANDS
0x03	NEG	-	Arithmetic negate.	0 Bytes (Relies on stack)
0x04	BIT_NOT	~	Bitwise NOT.	0 Bytes (Relies on stack)
0x05	LOG_NOT	!	Logical NOT.	0 Bytes (Relies on stack)

3. Arithmetic Operators

Binary operators. They pop two values from the stack, perform the calculation, and push the result. The instruction itself has no operands.

OPCODE	MNEMONIC	SYMBOL	DESCRIPTION	OPERANDS
0x06	DIV	/	Performs division.	0 Bytes
0x07	MOD	%	Performs modulo operation.	0 Bytes
0x08	ADD	+	Performs addition.	0 Bytes
0x09	SUB	-	Performs subtraction.	0 Bytes
0x0A	BIT_XOR	^	Performs bitwise XOR.	0 Bytes
0x0B	BIT_OR		Performs bitwise OR.	0 Bytes
0x0C	BIT_AND	&	Performs bitwise AND.	0 Bytes
0x13	MUL	*	Performs multiplication.	0 Bytes
0x16	SHR	>>	Performs bitwise right shift.	0 Bytes
0x17	SHL	<<	Performs bitwise left shift.	0 Bytes

4. Comparison Operators

Compares the two values at the top of the stack and pushes the boolean result. No inline operands.

OPCODE	MNEMONIC	SYMBOL	DESCRIPTION	OPERANDS
0x0D	EQ	==	Compares if equal.	0 Bytes
0x0F	GT	>	Compares if greater than.	0 Bytes
0x10	LT	<	Compares if less than.	0 Bytes
0x11	GE	>=	Compares if greater than or equal to.	0 Bytes
0x12	LE	<=	Compares if less than or equal to.	0 Bytes
0x14	NE	!=	Compares if not equal.	0 Bytes

5. Logical Operators

Logical calculation pops the final consolidated result to the stack. Instruction has no inline operands.

OPCODE	MNEMONIC	SYMBOL	DESCRIPTION	OPERANDS
0x0E	LOG_OR		Calculates logical OR (c = a b;) and pushes result to stack.	0 Bytes
0x15	LOG_AND	&&	Calculates logical AND (c = a && b;) and pushes result to stack.	0 Bytes

6. Flow Control & Structure

Contains jump, function call, wait, and other mechanisms. Most carry a 4-byte address/offset or ID.

OPCODE	MNEMONIC	SYMBOL	DESCRIPTION	OPERANDS (SIZE & TYPE)
0x18	JMP_FALSE		If the top stack value is false, jumps to specified address (used in if/while).	4 Bytes (Relative offset / Target address)
0x19	LINE_NUM	;	Marks the end of a statement and records the current line number of source code.	4 Bytes (Code line number)
0x1A	ARG_COMMIT	()	Commits arguments to non-extern functions. Does not appear alone; always pairs with CALL_FUNC.	0 Bytes
0x1B	AWAIT	await	Suspends script execution (async wait). Has a semicolon but doesn't record line number. The operand represents the offset of the statement inside the parentheses.	4 Bytes (Offset)
0x1C	JMP		Unconditional jump to target address (used in else/goto/break).	4 Bytes (Relative offset / Target address)
0x1D	IFF	iff	Executes special conditional check. Special format, followed by target addresses.	8 Bytes (Two 4-byte target addresses)
0x1E	CALL_FUNC	()	Executes a function.	4 Bytes (Function ID or address)
0x1F	RET	return	Terminates the current function, returning control or return value.	0 Bytes
0x2C	AND_JMP	&&	Short-circuit logic: If left operand is false, jumps immediately to failure label, skipping calculation of right operand early.	4 Bytes (Relative offset)
0x2D	OR_JMP		Short-circuit logic: If left operand is true, jumps immediately to success label, skipping calculation of right operand early.	4 Bytes (Relative offset)

7. Variables, Memory & Assignment

Except for array accesses that use an index from the stack, operations on variables mostly require a 4-byte Variable ID / Address.

OPCODE	MNEMONIC	SYMBOL	DESCRIPTION	OPERANDS (SIZE & TYPE)
0x20	POST_INC	i++	Variable postfix increment.	4 Bytes (Variable ID)
0x21	POST_DEC	i--	Variable postfix decrement.	4 Bytes (Variable ID)
0x22	PRE_INC	++i	Variable prefix increment.	4 Bytes (Variable ID)
0x23	PRE_DEC	--i	Variable prefix decrement.	4 Bytes (Variable ID)
0x24	PUSH_VAR		Pushes the value of a parameter or variable to the stack.	4 Bytes (Variable ID)
0x25	ADDR_OF	&	Gets the memory address of a variable.	4 Bytes (Variable ID)
0x26	ASSIGN	=	Assigns the value at top of stack to specified variable.	4 Bytes (Variable ID)
0x27	OP_ASSIGN		Compound assignment marker (specific operation decided by the immediate next byte, e.g., +=, -=).	5 Bytes (4-byte Variable ID + 1-byte opcode)
0x28	ARRAY_IDX	[]	Accesses an array element using the index popped from the stack.	0 Bytes (Relies on index popped from stack)

8. Type Casting

OPCODE	MNEMONIC	SYMBOL	DESCRIPTION	OPERANDS
0x29	CAST_STR	(string)	Converts current value at top of stack to string type.	0 Bytes

OPCODE	MNEMONIC	SYMBOL	DESCRIPTION	OPERANDS
0x2A	CAST_INT	(int)	Converts current value at top of stack to integer type.	0 Bytes
0x2B	CAST_SPR	(sprite)	[Removed] Converts current value at top of stack to a Sprite object.	0 Bytes

Call Mechanism and Special Structures

- When the virtual machine needs to manipulate a variable (e.g., `PUSH_VAR`), it uses the variable's physical index **P1_Index** in Part 1.
- When the virtual machine needs to call a function (e.g., `CALL_FUNC`), it uses the function's **Global ID** from the Symbol Table.
- The engine can only call functions that have been defined prior to the call; it cannot call functions defined after it. If such a call occurs, the syntax checker will report an `Undeclared Identifier` error.
- Every function always ends with a `RET`. They have no other meaning; they are engine placeholders used to mark the end of a function.
- During decompilation, decompilation is performed on a block-by-block basis using the function as the smallest unit. Jumps (`JMP`) across methods are impossible, because all invocations of other methods have dedicated opcodes such as `CALL_FUNC` and `call_ext_XX`.

IFF Structure

`iff` is followed by 4 + 4 operands. The first represents the address to jump to if it evaluates to false, and the second is the address where the evaluation condition starts (can be understood as the content inside the parentheses).

```

1  loc_0C1A:
2      0C1A:  24 00 00 00 00      PUSH_VAR    g_int
3      0C1F:  01 33 33 33 33      PUSH_INT    0x33333333
4      0C24:  0F                                GT
5      0C25:  1D 1D 00 00 00 0E      IFF          loc_0C43, loc_0C1A  ;
      Iff Jump
6          03 00 00
7      0C2E:  24 00 00 00 00      PUSH_VAR    g_int
8      0C33:  26 05 00 00 00      ASSIGN      sub_func_arg0
9      0C38:  1A                                ARG_COMMIT
10     0C39:  1E 04 00 00 00      CALL_FUNC    sub_func
11     0C3E:  19 4E 00 00 00      LINE_NUM     0x4E          ;
      Code Line 78
12  loc_0C43:

```

Corresponding source code:

```

1      iff (g_int > 0x33333333) {
2          sub_func(g_int);
3      }

```

Extern Function Calls

If the Opcode is greater than `0x40` (decimal 64), it means this is not a built-in instruction, but an **Extern function call**. **Instructions** `0x41 ~ 0xFF` (65~255) will map directly to the corresponding `call_ext_XX` method.

💡 Tip

Engine trivia: the number of exported functions in various engine versions is basically fixed, and they are generally identical.

The range is from 65 to 255.

I suspect they initially just allocated 1 byte of free space for them.

What happens when they run out of space? They threw all the rest into

`i_send_command.h` 🤪

The way it uses parameters is also different from regular functions that rely on `ARG_COMMIT`. It just pushes all the required parameters in and then calls.

```
1 CreateText(int font_vid,int x,int y,int z,string text,int
  behave=0)
2 {
3     int sprite;
4     sprite = CreateSprite(font_vid,x,y,z);
5     Action(sprite,ACT_SET_TEXT,&text);
6     Action(sprite,ACT_SET_BEHAVE,behave);
7     return sprite;
8 }
```

For instance, in this example, `CreateSprite` has 5 parameters but only 3 were provided, so during the call, two additional parameters belonging to `CreateSprite` (where default values exist) will be pushed in to fill the space.

```
1 ;Function CreateText() Begin
2   C96A: 19 CD 03 00 00      LINE_NUM      0x3CD          ;
   Code Line 973
3   C96F: 24 4B 01 00 00      PUSH_VAR      CreateText_arg0
4   C974: 24 4C 01 00 00      PUSH_VAR      CreateText_arg1
5   C979: 24 4D 01 00 00      PUSH_VAR      CreateText_arg2
6   C97E: 24 4E 01 00 00      PUSH_VAR      CreateText_arg3
7   C983: 24 05 00 00 00      PUSH_VAR      CreateSprite_arg4
8   C988: 24 06 00 00 00      PUSH_VAR      CreateSprite_arg5
9   C98D: 41                  CALL_EXT_65      ;
   CreateSprite
10  C98E: 26 51 01 00 00      ASSIGN        CreateText_local0
11  C993: 19 CE 03 00 00      LINE_NUM      0x3CE          ;
   Code Line 974
12 ; ...
13  C9E1: 1F                  RET              ;
   mark end of function
14 ;Function CreateText() End
15 ; -----
```

Local Non-static Variable Initialization

Remember what was mentioned at the beginning about non-static variable initialization?

Here comes the most bizarre/abstract part.

```
1 ScriptEvent4( int unit, int var1 = 114, string var2 = "text" ){
2     static int stval = 100;
3     static int starr[] = {0x2222, 0x3333, 0x4444, 0x5555};
4     int val = 100;
5     int arr[] = {0x2222, 0x3333, 0x4444, 0x5555};
6 }
7
```

Non-arrays are okay, just a pretty normal push+assign.

```
1 0B9D: 01 64 00 00 00 PUSH_INT 0x64
2 0BA2: 26 1C 00 00 00 ASSIGN ScriptEvent4_local5
3 0BA7: 19 16 00 00 00 LINE_NUM 0x16 ;
Code Line 22
4 0BAC: 19 17 00 00 00 LINE_NUM 0x17 ;
Code Line 23
```

Arrays... well, it's hard to judge. It treats each element in it as a separate int, assigns a value to it individually, and reassembles it when the assignments are done.

1	0BB1:	01 22 22 00 00	PUSH_INT	0x2222	
2	0BB6:	26 1D 00 00 00	ASSIGN	ScriptEvent4_local6	
3	0BBB:	19 17 00 00 00	LINE_NUM	0x17	;
Code Line 23					
4	0BC0:	01 33 33 00 00	PUSH_INT	0x3333	
5	0BC5:	26 1E 00 00 00	ASSIGN	ScriptEvent4_local7	
6	0BCA:	19 17 00 00 00	LINE_NUM	0x17	;
Code Line 23					
7	0BCF:	01 44 44 00 00	PUSH_INT	0x4444	
8	0BD4:	26 1F 00 00 00	ASSIGN	ScriptEvent4_local8	
9	0BD9:	19 17 00 00 00	LINE_NUM	0x17	;
Code Line 23					
10	0BDE:	01 55 55 00 00	PUSH_INT	0x5555	
11	0BE3:	26 20 00 00 00	ASSIGN	ScriptEvent4_local9	
12	0BE8:	19 17 00 00 00	LINE_NUM	0x17	;
Code Line 23					

Stack Mechanism and Derived Bugs

As mentioned earlier, the entire LGD script executes within the virtual machine based on a **Stack** architecture.

Working Principle of the Virtual Machine Stack

When executing bytecode, the engine uses a Last-In, First-Out (LIFO) evaluation stack:

- **Push (PUSH):** When encountering constants (e.g., `PUSH_INT`) or variables (e.g., `PUSH_VAR`), the VM pushes their values onto the top of the stack.
- **Calculate & Pop (POP):** When encountering operators (such as arithmetic, comparison, and logical operators), the VM pops the required number of operands from the top of the stack, performs the calculation, and pushes the result back onto the stack.
 - *For example:* To compute `a + b`, it first PUSHes `a`, then PUSHes `b`, and then encounters the `ADD` instruction, which pops both, adds them, and pushes the sum back onto the stack.
- **Extern Calls:** When calling an Extern function, the VM sequentially pushes the

parameters onto the stack, which are then popped and used by the function body all at once.

Classic Stack-related Bugs

Since it is a stack-based VM, classic stack-related bugs inevitably occur:

1. **Stack Underflow**: The VM attempts to execute an instruction that requires operands, but there is insufficient data in the stack (e.g., executing `ADD` when the stack is completely empty). This is rare, as the compiler usually rejects such code.
2. **Stack Overflow**: Pushing too much data onto the stack in a short time, exceeding the maximum stack depth of the VM. This is also quite rare.
3. **Stack Imbalance (Stack Out / Stack In)**: A hidden bug. The compilation passes, and sometimes the game even runs fine (this is even visible in official scripts). It is caused by lenient validation during compiler design.

Stack Out Triggered by Orphaned Expressions

Source Code

From the AS-Free 4.5.3 source code:

```
1 // Certain specific menus do not require popping up a level-up
  confirmation dialog
2 isSkipConfirmationLevelUpFor(int a_currentMenu)
3 {
4     if ( (currentMenu == MENU_CHARACTERS_INFO)
5         || (currentMenu == MENU_CHARACTER_SKILLS)
6         || (currentMenu == MENU_SHOP)
7         || (currentMenu == MENU_INVENTORY)
8         || (currentMenu == MENU_MISSION_INFO)
9         )
10        true; // ⚠ Likely a simple omission of a 'return'
              statement, who knows ⚠
11        // Although syntactically valid and compiles without
              errors, it has no practical effect.
```

```

12
13     return false;
14 }

```

Assembly Code

```

1  ;Function isSkipConfirmationLevelUpFor() Begin
2  556DF: 24 E7 0A 00 00    PUSH_VAR    currentMenu
3  556E4: 01 73 00 00 00    PUSH_INT    115
4  556E9: 0D                EQ
5  556EA: 2D 10 00 00 00    OR_JMP      loc_556FB    ;
    Short-circuit logical jump
6  ;... (Omitted multiple logical OR operations in between) ...
7  loc_5572E:
8  5572E: 18 0E 00 00 00    JMP_FALSE   loc_5573D    ;
    If 'if' condition is false, skip
9  55733: 01 01 00 00 00    PUSH_INT    1            ;
    ⚠ Push operation generated by compiling 'true;'. This '1' is
    never used afterward
10 55738: 19 0F 00 00 00    LINE_NUM    15           ;
    Code Line 15
11 loc_5573D:
12 5573D: 01 00 00 00 00    PUSH_INT    0            ;
    Push return value '0' of 'return false'
13 55742: 1F                RET                      ;
    Execute return, pop '0' from stack top and return
14 55743: 19 11 00 00 00    LINE_NUM    17
15 55748: 1F                RET                      ;
    mark end of function
16 ;Function isSkipConfirmationLevelUpFor() End

```

Decompiled Code

```
1  isSkipConfirmationLevelUpFor(int
   isSkipConfirmationLevelUpFor_arg0)
2  {
3      if ((((((currentMenu == 115) || (currentMenu == 117)) ||
   (currentMenu == 104)) || (currentMenu == 118)) || (currentMenu ==
   100)))) {
4          // WARNING: Orphaned expression -> Stack_Out = 1
5          // --- Line 15 ---
6      }
7      return 0;
8      // --- Line 17 ---
9  }
```

Bug Mechanism

1. When the `if` condition evaluates to FALSE:

- It jumps directly to `loc_5573D`, pushes `0`, and executes `RET` to pop `0` and return normally. At this point, the stack is balanced.

2. When the `if` condition evaluates to TRUE:

- It executes `PUSH_INT 1`, pushing `1` onto the top of the stack.
- Since there is no `else` block, the program sequentially proceeds to `loc_5573D` and executes `PUSH_INT 0` again.
- At this point, **both `1` and `0` coexist in the stack!**
- The `RET` instruction is executed immediately, and the VM pops `0` from the top of the stack as the function's return value.
- However, **the previously pushed `1` is forgotten and left in the stack** without being cleared.

This "forgotten 1" is the `Orphaned expression -> Stack_Out = 1` (residual stack value left by an orphaned expression) warned about by the decompiler. If this function is repeatedly called in the game's main loop, the VM's evaluation stack will eventually overflow, causing the game to crash.

Stack In Triggered by Hanging Operators

In official scripts, you may also encounter another bug related to `#ifdef`.

This bug is even less common; it was only encountered once during testing of AS-Free 2.0.4. Unfortunately, that version did not provide source code, so we can only deduce the scenario using a simulated test script.

Source Code

```
1 test_ifdef_bug_and_end()
2 {
3     int money = 10;
4
5     // If TARGET_PLATFORM_IOS is not defined
6     if ( money > 100 &&
7 #ifdef TARGET_PLATFORM_IOS
8         CheckIOSPurchase()
9 #endif
10    )
11    {
12        money = 0;
13    }
14 }
```

In a proper C++ compiler, if `TARGET_PLATFORM_IOS` is undefined, the preprocessing expansion results in `if ( money > 100 && )`, which is rejected at compile time due to the missing right operand.

However, the simple LGC compiler **successfully compiled it** and generated the following bizarre bytecode:

Assembly Code

```
1 ;Function test_ifdef_bug_and_end() Begin
2 09DA: 01 0A 00 00 00    PUSH_INT    10
3 09DF: 26 00 00 00 00    ASSIGN
test_ifdef_bug_and_end_local0
```

```

4  09E4: 19 11 00 00 00    LINE_NUM    17
5  09E9: 19 12 00 00 00    LINE_NUM    18
6  09EE: 24 00 00 00 00    PUSH_VAR
   test_ifdef_bug_and_end_local0 ; Push money (10)
7  09F3: 01 64 00 00 00    PUSH_INT    100
   ; Push 100
8  09F8: 0F                                GT
   ; Evaluate money > 100 (result is false/0)
9  09F9: 2C 05 00 00 00    AND_JMP     loc_09FF          ;
   ⚠ Short-circuit AND: If top of stack is false, jump to loc_09FF
10 09FE: 15                                LOG_AND          ;
   ⚠ Logical AND: Requires popping two operands!
11 loc_09FF:
12 09FF: 18 13 00 00 00    JMP_FALSE   loc_0A13          ;
   If condition is false, skip the 'if' block
13 0A04: 01 00 00 00 00    PUSH_INT    0
14 0A09: 26 00 00 00 00    ASSIGN
   test_ifdef_bug_and_end_local0
15 loc_0A13:
16 0A13: 1F                                RET
17 ;Function test_ifdef_bug_and_end() End

```

Bug Mechanism

The LGC compiler employs **Short-Circuit Evaluation** for the logical AND (&&):

1. **AND_JMP (Opcode 2C)**: If the left-hand condition is false (0), it immediately short-circuits and jumps to `loc_09FF`, terminating the subsequent evaluation of `&&` early.
2. **LOG_AND (Opcode 15)**: If the left-hand condition is true (1), it continues evaluating the subsequent expression and eventually calls `LOG_AND` to pop two values from the top of the stack for a logical AND.

But since the content inside the `#ifdef` block was completely hollowed out, there was no executable expression following the `&&`. Consequently, something extremely preposterous happened:

- **Path A: When `money` is 10 (`money > 100` is FALSE)**

- a. The `GT` result evaluates to `0` and is pushed onto the stack.
 - b. `AND_JMP` sees `0` on the stack top and **triggers the short-circuit jump**, leaping directly to `loc_09FF`!
 - c. It successfully **bypasses** the underlying `09FE: 15 LOG_AND` bytecode.
 - d. Then, `JMP_FALSE` reads `0` from the stack top and skips the `if` block.
 - e. **Result:** Because the short-circuit jump bypassed the `LOG_AND` instruction that lacked operands, the stack structure did not break, and the program miraculously executed perfectly! This is the so-called **"Stack In" lucky success state**.
- **Path B: When `money` is set to 200 (`money > 100` is TRUE)**
 - a. The `GT` result evaluates to `1` and is pushed onto the stack.
 - b. `AND_JMP` reads `1`, **does not trigger the jump**, and the program is forced to execute sequentially.
 - c. The VM executes the next instruction: `09FE: 15 LOG_AND`.
 - d. `LOG_AND` is a binary operator that attempts to **pop two values** from the stack for a logical AND.
 - e. However, the stack contains only the computed `1` (because the subsequent expression was gutted, nothing was ever pushed onto the stack as the second value)!
 - f. **Result:** With only one operand in the stack, `LOG_AND` forces a pop, triggering a Stack Underflow, or pops unrelated data from the bottom of the stack, leading to an unpredictable crash of the VM!

The exact same logic applies to the logical OR (`||`), corresponding to the short-circuit jump `OR_JMP` and the binary operator `LOG_OR`.

Universality of Hanging Binary Operators and Compiler Syntax Validation Rules

It is crucial to emphasize that this "hanging operator" vulnerability is by no means limited to the logical `&&` and `||` operators; it applies to all binary operators (such as `+`, `-`, `*`, `/`, `==`, `>`, `<`, etc.).

The rules are as follows:

Rule A: Compilation Permitted (Operator Adjacent to Parentheses)

When a hanging binary operator is adjacent to a delimiter (parentheses `(` or `)`) or nested, the compiler directly deems the syntax valid and proceeds:

Regardless of what happens, the compilation succeeds, and decompilation always produces a `stack_in` warning.

- **Operator adjacent to a right parenthesis:** `if ( A && )` ✓
- **Operator adjacent to a left parenthesis:** `if ( || B )` ✓
- **Hanging in nested parentheses:** `if ( (A || B) && )` ✓

Underlying Logic:

The compiler's syntax tree is imperfect. It only validates the pairing of parentheses and the overall boundaries of the expression, but **completely fails to verify whether both sides of a binary operator are connected to valid sub-expressions or operands**. As long as it encounters parentheses, it assumes it has reached the boundary of the expression, completely ignoring the flaw of the hanging operator and generating the corresponding binary calculation bytecode (such as `LOG_AND`, `LOG_OR`, `ADD`, etc.) as usual.

Rule B: Compilation Rejected with Error (Operator Adjacent to Operator)

If two binary operators appear immediately adjacent to each other in an expression—for example, due to consecutive hanging `#ifdef` blocks—the compiler will directly throw an error:

- **Different adjacent operators:** `if ( A && || B )` ✗
- **Identical adjacent operators:** `if ( A && && )` ✗

Underlying Logic:

This is because during the lexical analysis (Lexing) or basic token stream validation stage, the compiler enforces an ironclad rule: "The next non-empty token after a binary operator must never be another binary operator." When two binary operators appear consecutively, the compiler cannot even generate a basic syntax tree and directly throws an `Undeclared Identifier` or lexical parsing error.